

Plano de Projeto — Renovação do Ascend

Versão: 1.0

Status: baseline para planejamento

Produto: Ascend

Plataformas iniciais: Android e iOS

Idioma inicial: Português brasileiro

Arquitetura atual: Flutter, Riverpod, Isar, Firebase Auth, Firestore e backend Java/Spring Boot

Decisão consolidada: funcionalidades competitivas não fazem parte do produto atual.

1. Resumo executivo

O Ascend será reposicionado como um aplicativo de evolução pessoal baseado em progressão de RPG.

O usuário transforma objetivos reais em Jornadas, divide essas Jornadas em missões, desenvolve atributos, constrói uma build, enfrenta desafios semanais e registra sua evolução ao longo do tempo.

A retirada da competição elimina a necessidade de comparar atividades diferentes, validar evidências, impedir fraudes competitivas e manter rankings globais. A arquitetura autoritativa do backend, contudo, deve ser preservada para garantir consistência, sincronização e integridade da progressão pessoal.

O foco da renovação será fortalecer cinco elementos:

1. Uso diário simples.
 2. Progressão pessoal significativa.
 3. Objetivos estruturados como Jornadas.
 4. Desafios semanais e Provas de Ascensão.
 5. Confiabilidade suficiente para produção.
-

2. Contexto do projeto

O Ascend já possui uma base funcional e arquitetural voltada para:

- autenticação;
- missões;
- progressão;
- atributos;
- níveis;
- chefe semanal;
- armazenamento local;
- sincronização;

- backend autoritativo.

A competição foi removida porque sua implementação exigia resolver problemas desproporcionais para a primeira versão:

- comparação entre atividades diferentes;
- comprovação de ações reais;
- integração com fontes externas;
- prevenção de fraude;
- moderação;
- balanceamento de recompensas;
- contestação de resultados;
- manutenção de rankings e temporadas.

A retirada dessa área torna necessário reforçar outras partes do produto para que o Ascend não seja percebido apenas como um gerenciador de tarefas gamificado.

3. Aprendizados de mercado

As avaliações analisadas do concorrente direto Arise demonstram que existe interesse real em:

- interfaces que transmitam a sensação de um “Sistema”;
- níveis;
- atributos;
- personagem;
- missões;
- progressão inspirada em RPG;
- transformação pessoal apresentada como uma jornada.

Ao mesmo tempo, aparecem críticas frequentes relacionadas a:

- paywall apresentado somente depois de um onboarding longo;
- ausência de uma versão gratuita utilizável;
- falta de português;
- solicitações de avaliação antes do uso;
- personalização apenas aparente;
- recomendações incompatíveis com as restrições informadas;
- dificuldade para editar planos;
- bugs que bloqueiam o fluxo principal;
- repetição de configurações;
- progressão sem impacto prático.

Esses problemas deverão ser tratados como requisitos de produto, e não apenas como oportunidades de marketing.

4. Visão do produto

O Ascend transforma objetivos pessoais em Jornadas e ações reais em evolução de personagem.

O usuário deve conseguir:

- definir para onde deseja evoluir;
 - visualizar o que precisa fazer agora;
 - entender como cada ação contribui para sua evolução;
 - tomar decisões sobre sua build;
 - enfrentar desafios proporcionais à própria realidade;
 - recuperar-se de períodos de baixa consistência;
 - consultar a história de sua evolução.
-

5. Proposta de valor

Complete missões, desenvolva seus atributos, construa sua build e avance por Jornadas criadas a partir dos seus objetivos reais.

5.1 Diferenciais pretendidos

O Ascend deverá diferenciar-se por:

- português brasileiro desde a concepção;
 - plano gratuito funcional;
 - progressão explicável;
 - Jornadas conectadas a objetivos de médio prazo;
 - personalização editável;
 - recuperação sem punição destrutiva;
 - RPG integrado à estrutura do produto;
 - funcionamento sem dependência obrigatória de IA;
 - ausência de verificações invasivas;
 - experiência confiável online e offline.
-

6. Objetivos do projeto

6.1 Objetivo principal

Preparar o Ascend para uma primeira versão de produção com um loop pessoal de evolução completo, estável e diferenciável.

6.2 Objetivos específicos

- reconstruir o loop diário;
- simplificar o onboarding;
- fortalecer a gestão de missões;
- tornar a progressão compreensível;
- criar Jornadas;
- implementar build e talentos;
- reformular o chefe semanal;
- implementar recuperação de consistência;
- substituir promoções competitivas por Provas de Ascensão;
- criar o Legado;
- preparar monetização transparente;
- garantir qualidade técnica para produção.

6.3 Não objetivos

Não fazem parte desta versão:

- rankings globais;
- temporadas competitivas;
- quests competitivas;
- validação por câmera;
- Health Connect obrigatório;
- Strava;
- comparação pública entre usuários;
- anti-cheat competitivo;
- recomendação médica;
- prescrição automática de treinos;
- rede social aberta;
- marketplace;
- economia complexa de equipamentos.

7. Público inicial

O público prioritário é formado por pessoas que:

- gostam de jogos e sistemas de progressão;
- desejam desenvolver disciplina;
- têm dificuldade para manter hábitos;
- possuem objetivos pessoais, acadêmicos ou profissionais;
- consideram listas tradicionais de tarefas pouco motivadoras;
- querem visualizar sua evolução;
- utilizam o celular como principal ferramenta de organização.

7.1 Necessidades principais

O usuário precisa:

- saber o que fazer agora;
 - entender por que aquela ação é relevante;
 - perceber evolução;
 - reorganizar-se após falhar;
 - não perder meses de progresso por uma pausa;
 - adaptar o plano à própria rotina;
 - manter controle sobre sugestões automáticas.
-

8. Princípios de produto

8.1 A ação real possui prioridade

O Ascend deve incentivar atividades fora do aplicativo. O produto não será otimizado apenas para aumentar tempo de tela.

8.2 O RPG deve possuir consequência

XP, atributos, níveis, builds e talentos precisam alterar ou desbloquear elementos da experiência.

8.3 O usuário mantém o controle

Nenhuma missão sugerida deve ser obrigatória. O usuário poderá editar, substituir, reagendar, pausar ou arquivar.

8.4 Falha não apaga progresso

Ausências podem afetar o Momentum, mas não devem remover níveis, atributos, títulos ou conquistas permanentes.

8.5 Configuração progressiva

O usuário deve utilizar o produto antes de conhecer todas as configurações disponíveis.

8.6 Transparência comercial

O plano gratuito e o plano pago devem ser explicados antes de qualquer fluxo longo de personalização.

8.7 IA opcional

Nenhum fluxo essencial dependerá de serviços generativos.

8.8 Identidade original

A interface pode utilizar conceitos de RPG e “Sistema”, mas não deve reproduzir diretamente:

- assets;
 - personagens;
 - nomes;
 - símbolos;
 - tipografia;
 - efeitos;
 - interfaces;
 - elementos narrativos protegidos de animes ou jogos existentes.
-

9. Estrutura conceitual

O Ascend será estruturado em três camadas.

9.1 Camada prática

Responsável pela organização:

- missões;
- recorrências;
- lembretes;
- subtarefas;
- planejamento;
- histórico;
- reagendamento;
- sincronização.

9.2 Camada motivacional

Responsável pela continuidade:

- feedback de conclusão;
- Momentum;
- chefe semanal;
- revisão;
- próximos desbloqueios;
- retomada.

9.3 Camada narrativa

Responsável pelo diferencial:

- personagem;
 - atributos;
 - build;
 - talentos;
 - Jornadas;
 - Provas;
 - Patamares;
 - Legado.
-

10. Loop principal

```
Definir um objetivo
    ↓
Iniciar uma Jornada
    ↓
Planejar missões
    ↓
Executar ações reais
    ↓
Receber XP e progressão
    ↓
Desenvolver atributos e build
    ↓
Enfrentar o chefe semanal
    ↓
Revisar a semana
    ↓
Concluir uma Prova de Ascensão
    ↓
Avançar de Patamar
```

10.1 Loop diário

```
Abrir o aplicativo
    ↓
Ver a missão recomendada
    ↓
Executar, reagendar ou substituir
```

↓
Registrar a conclusão
↓
Receber feedback
↓
Ver o próximo passo

11. Arquitetura da informação

A navegação principal terá quatro áreas.

11.1 Base

Responde:

Como estou evoluindo?

Conteúdo:

- personagem;
- nível;
- XP;
- Momentum;
- build;
- atributos;
- próximo desbloqueio;
- resumo da Jornada;
- situação do chefe semanal.

11.2 Missões

Responde:

O que preciso fazer agora?

Conteúdo:

- missão recomendada;
- missões de hoje;
- missões em andamento;
- rotinas;
- concluídas;
- criação rápida;

- filtros.

11.3 Jornada

Responde:

Para onde estou indo?

Conteúdo:

- objetivo;
- motivação;
- capítulos;
- marcos;
- progresso;
- missões relacionadas;
- próximo marco.

11.4 Ascensão

Responde:

Qual desafio estou enfrentando?

Conteúdo:

- chefe semanal;
- Prova de Ascensão;
- Patamar;
- títulos;
- recordes pessoais;
- Legado.

11.5 Perfil

Acessível fora da navegação principal:

- conta;
- aparência;
- notificações;
- assinatura;
- suporte;
- privacidade;
- exportação;
- exclusão da conta.

12. Fluxos principais

12.1 Primeiro acesso

1. Apresentação da proposta.
2. Explicação resumida do plano gratuito.
3. Escolha do objetivo principal.
4. Escolha da área inicial.
5. Definição de disponibilidade.
6. Criação do personagem.
7. Escolha ou criação da primeira Jornada.
8. Geração de até três missões.
9. Revisão das missões.
10. Primeira conclusão.
11. Exibição do ganho de XP.
12. Apresentação da Base.

12.2 Criação de missão

1. Usuário informa nome.
2. Seleciona quando pretende realizá-la.
3. Seleciona a área.
4. O sistema sugere atributo e dificuldade.
5. Usuário confirma ou edita.
6. Missão é criada.
7. Ocorrências são geradas quando necessário.

12.3 Conclusão

1. Usuário conclui a missão.
2. O cliente registra a ação localmente.
3. O backend valida a conclusão.
4. O backend calcula recompensas.
5. O evento de progressão é persistido.
6. A interface mostra o resultado.
7. O cache local é reconciliado.

12.4 Retorno após ausência

1. O sistema identifica pendências acumuladas.
2. Nenhuma missão vencida é automaticamente apresentada como culpa.
3. O usuário escolhe entre:
 4. retomada leve;

5. manutenção do plano;
6. reorganização da Jornada.
7. O sistema ajusta o plano.
8. O Momentum passa para “retomando”.

13. Requisitos funcionais

As prioridades serão classificadas em:

- **P0:** necessário para lançamento;
- **P1:** necessário após estabilização;
- **P2:** evolução futura.

13.1 Conta e autenticação

ID	Requisito	Prioridade
RF-AUT-01	Permitir criação e autenticação de conta	P0
RF-AUT-02	Restaurar automaticamente a sessão válida	P0
RF-AUT-03	Não exigir repetição do onboarding após novo login	P0
RF-AUT-04	Permitir recuperação de acesso	P0
RF-AUT-05	Permitir logout	P0
RF-AUT-06	Permitir exclusão da conta e dos dados associados	P0
RF-AUT-07	Permitir exportação dos dados pessoais	P1

Critérios de aceite

- O usuário autenticado não deve responder novamente ao onboarding.
- Falhas temporárias de rede não devem destruir a sessão local.
- A exclusão deve exigir confirmação explícita.
- Dados locais devem ser limpos após exclusão confirmada.

13.2 Onboarding

ID	Requisito	Prioridade
RF-ONB-01	Explicar a proposta do Ascend	P0

ID	Requisito	Prioridade
RF-ONB-02	Informar claramente o que é gratuito e pago	P0
RF-ONB-03	Coletar somente informações necessárias para iniciar	P0
RF-ONB-04	Persistir cada etapa concluída	P0
RF-ONB-05	Permitir retomar o onboarding após interrupção	P0
RF-ONB-06	Criar uma primeira Jornada	P0
RF-ONB-07	Criar ou sugerir até três missões iniciais	P0
RF-ONB-08	Permitir editar todas as sugestões antes de ativá-las	P0
RF-ONB-09	Não solicitar avaliação da loja	P0
RF-ONB-10	Permitir pular informações opcionais	P0

Critérios de aceite

- O usuário deve chegar à primeira missão sem preencher questionários extensos.
- Nenhum paywall deve bloquear o primeiro contato com o loop principal.
- Interromper o aplicativo não deve reiniciar o fluxo.

13.3 Missões

ID	Requisito	Prioridade
RF-MIS-01	Criar missão única	P0
RF-MIS-02	Criar missão recorrente	P0
RF-MIS-03	Editar missão antes da conclusão	P0
RF-MIS-04	Pausar missão recorrente	P0
RF-MIS-05	Arquivar missão	P0
RF-MIS-06	Reagendar ocorrência	P0
RF-MIS-07	Concluir missão	P0
RF-MIS-08	Revogar conclusão	P0
RF-MIS-09	Adicionar subtarefas	P1
RF-MIS-10	Adicionar lembretes	P0
RF-MIS-11	Vincular missão a uma Jornada	P0
RF-MIS-12	Associar atributo principal	P0

ID	Requisito	Prioridade
RF-MIS-13	Sugerir dificuldade	P0
RF-MIS-14	Permitir concluir offline	P0
RF-MIS-15	Exibir histórico	P0
RF-MIS-16	Filtrar missões	P1
RF-MIS-17	Criar sessão de foco	P1
RF-MIS-18	Criar desafio semanal	P1

Regra de recorrência

A definição da missão deverá ser separada da ocorrência.

Exemplo:

Missão recorrente: Estudar inglês
Ocorrência: Estudar inglês em 14/07/2026

Isso permitirá editar recorrências sem alterar incorretamente o histórico.

Critérios de aceite

- Uma conclusão não pode gerar recompensa duplicada.
- Revogar uma conclusão deve produzir um evento compensatório.
- Editar a recorrência não deve apagar ocorrências concluídas.
- Uma missão criada offline deve ser sincronizada posteriormente.

13.4 Jornadas

ID	Requisito	Prioridade
RF-JOR-01	Criar Jornada manualmente	P0
RF-JOR-02	Criar Jornada a partir de modelo	P0
RF-JOR-03	Definir objetivo e motivação	P0
RF-JOR-04	Definir capítulos	P0
RF-JOR-05	Definir marcos	P0
RF-JOR-06	Vincular missões	P0
RF-JOR-07	Exibir progresso	P0

ID	Requisito	Prioridade
RF-JOR-08	Concluir capítulo	P0
RF-JOR-09	Concluir Jornada	P0
RF-JOR-10	Pausar Jornada	P0
RF-JOR-11	Arquivar Jornada	P0
RF-JOR-12	Manter múltiplas Jornadas ativas	P1
RF-JOR-13	Recomendar ajustes na Jornada	P1

Critérios de aceite

- Uma Jornada não pode ser reduzida apenas à contagem de tarefas.
- Marcos obrigatórios e opcionais devem ser diferenciados.
- A conclusão deve gerar registro no Legado.
- O usuário deve poder continuar consultando Jornadas concluídas.

13.5 Progressão

ID	Requisito	Prioridade
RF-PRG-01	Conceder XP geral	P0
RF-PRG-02	Evoluir nível	P0
RF-PRG-03	Conceder XP de atributo	P0
RF-PRG-04	Exibir origem da recompensa	P0
RF-PRG-05	Exibir próximo desbloqueio	P0
RF-PRG-06	Prevenir duplicação de recompensa	P0
RF-PRG-07	Aplicar limites contra exploração	P0
RF-PRG-08	Manter histórico de progressão	P0
RF-PRG-09	Compensar recompensas revogadas	P0

Exemplo de feedback

Missão concluída: Revisar capítulo do TCC

+40 XP geral
+25 Intelecto
+10 Disciplina

+12 de dano no chefe

Jornada "Concluir o TCC": 38%

Critérios de aceite

O usuário deve conseguir identificar:

- qual ação gerou a recompensa;
- quanto recebeu;
- quais atributos foram afetados;
- qual progresso foi alterado;
- o que está próximo de desbloquear.

13.6 Atributos

Conjunto inicial proposto:

- Força;
- Vitalidade;
- Intelecto;
- Disciplina;
- Agilidade;
- Conexão.

ID	Requisito	Prioridade
RF-ATR-01	Exibir atributos do personagem	P0
RF-ATR-02	Vincular missões a atributos	P0
RF-ATR-03	Evoluir atributo por eventos válidos	P0
RF-ATR-04	Exibir histórico do atributo	P1
RF-ATR-05	Explicar o significado de cada atributo	P0
RF-ATR-06	Limitar crescimento artificial repetitivo	P0

13.7 Build e talentos

ID	Requisito	Prioridade
RF-BLD-01	Permitir escolher uma build	P0
RF-BLD-02	Disponibilizar ao menos três builds	P0

ID	Requisito	Prioridade
RF-BLD-03	Exibir identidade e vantagens da build	P0
RF-BLD-04	Conceder pontos de talento	P0
RF-BLD-05	Permitir desbloquear talentos	P0
RF-BLD-06	Aplicar efeitos dos talentos	P0
RF-BLD-07	Permitir redefinição controlada	P1
RF-BLD-08	Disponibilizar builds adicionais	P1

Builds iniciais sugeridas

Erudito

Foco em:

- estudo;
- leitura;
- aprendizagem;
- sessões de foco.

Vanguarda

Foco em:

- ação;
- atividade física;
- vitalidade;
- execução.

Estrategista

Foco em:

- organização;
- planejamento;
- consistência;
- conclusão de projetos.

Regra

A build deve ser escolhida pelo usuário. Não será definida automaticamente pelo maior atributo.

13.8 Momentum

ID	Requisito	Prioridade
RF-MOM-01	Calcular Momentum atual	P0
RF-MOM-02	Exibir estado do Momentum	P0
RF-MOM-03	Considerar frequência planejada	P0
RF-MOM-04	Considerar consistência recente	P0
RF-MOM-05	Reduzir Momentum gradualmente	P0
RF-MOM-06	Não remover progressão permanente	P0
RF-MOM-07	Reconhecer retomada	P0

Estados sugeridos:

- adormecido;
- retomando;
- estável;
- crescente;
- imparável.

13.9 Chefe semanal

ID	Requisito	Prioridade
RF-BOS-01	Criar um chefe por ciclo semanal	P0
RF-BOS-02	Calcular vida com base no planejamento	P0
RF-BOS-03	Causar dano por missões concluídas	P0
RF-BOS-04	Limitar contribuição de missões tardias	P0
RF-BOS-05	Aplicar combos de consistência	P1
RF-BOS-06	Conceder recompensa por vitória	P0
RF-BOS-07	Gerar revisão em caso de derrota	P0
RF-BOS-08	Registrar chefe no Legado	P0
RF-BOS-09	Exibir histórico de chefes	P1

Critérios de aceite

- A vida do chefe deve ser proporcional ao plano do usuário.

- Criar várias missões depois de quase derrotá-lo não pode gerar vantagem integral.
- A derrota não remove XP, nível ou atributos.
- O usuário deve receber opções de ajuste para a semana seguinte.

13.10 Recuperação

ID	Requisito	Prioridade
RF-REC-01	Detectar ausência relevante	P0
RF-REC-02	Oferecer retomada leve	P0
RF-REC-03	Permitir arquivamento em lote	P0
RF-REC-04	Permitir redistribuir missões	P0
RF-REC-05	Preservar progresso permanente	P0
RF-REC-06	Ajustar Momentum para retomada	P0
RF-REC-07	Permitir pausa planejada	P1

13.11 Provas de Ascensão

ID	Requisito	Prioridade
RF-PRO-01	Gerar Prova a partir do progresso	P0
RF-PRO-02	Exibir critérios objetivos	P0
RF-PRO-03	Permitir iniciar Prova	P0
RF-PRO-04	Exibir progresso	P0
RF-PRO-05	Concluir Prova	P0
RF-PRO-06	Permitir nova tentativa	P0
RF-PRO-07	Conceder recompensa permanente	P0

Exemplos:

- completar cinco sessões de foco em sete dias;
- cumprir três rotinas essenciais por duas semanas;
- concluir um capítulo;
- derrotar determinada quantidade de chefes;
- executar uma missão previamente classificada como difícil.

13.12 Patamares

Patamares iniciais sugeridos:

1. Desperto;
2. Iniciado;
3. Adepto;
4. Especialista;
5. Mestre;
6. Ascendente.

ID	Requisito	Prioridade
RF-PAT-01	Exibir Patamar atual	P0
RF-PAT-02	Exibir critérios do próximo Patamar	P0
RF-PAT-03	Exigir Prova para promoção	P0
RF-PAT-04	Manter Patamar permanentemente	P0
RF-PAT-05	Registrar promoção no Legado	P0

13.13 Legado

ID	Requisito	Prioridade
RF-LEG-01	Registrar Jornadas concluídas	P0
RF-LEG-02	Registrar chefes derrotados	P0
RF-LEG-03	Registrar Provas concluídas	P0
RF-LEG-04	Registrar Patamares	P0
RF-LEG-05	Registrar títulos	P1
RF-LEG-06	Exibir linha do tempo	P1
RF-LEG-07	Exibir evolução histórica	P1

13.14 Notificações

ID	Requisito	Prioridade
RF-NOT-01	Lembrar missão planejada	P0
RF-NOT-02	Permitir configurar horários	P0

ID	Requisito	Prioridade
RF-NOT-03	Permitir desativar categorias	P0
RF-NOT-04	Evitar notificações excessivas	P0
RF-NOT-05	Notificar sobre encerramento semanal	P1
RF-NOT-06	Notificar sobre Prova ativa	P1

13.15 Monetização

ID	Requisito	Prioridade
RF-MON-01	Permitir uso gratuito contínuo	P0
RF-MON-02	Informar limitações antes do onboarding longo	P0
RF-MON-03	Exibir preço localizado	P0
RF-MON-04	Restaurar compra	P0
RF-MON-05	Reconhecer assinatura entre dispositivos	P0
RF-MON-06	Manter dados após cancelamento	P0
RF-MON-07	Não exigir convite para teste	P0
RF-MON-08	Disponibilizar múltiplas Jornadas no Premium	P1
RF-MON-09	Disponibilizar análises avançadas no Premium	P1
RF-MON-10	Disponibilizar IA opcional no Premium	P1

14. Requisitos não funcionais

14.1 Confiabilidade

ID	Requisito
RNF-CON-01	Operações de recompensa devem ser idempotentes
RNF-CON-02	Dados não podem desaparecer após atualização
RNF-CON-03	Onboarding deve ser persistido por etapa
RNF-CON-04	Missões offline devem ser reconciliadas
RNF-CON-05	Falha de IA não pode bloquear o produto

ID	Requisito
RNF-CON-06	Falha temporária de backend deve permitir nova tentativa
RNF-CON-07	O aplicativo deve possuir estados de erro recuperáveis

14.2 Desempenho

- A Base deve carregar inicialmente a partir do cache local.
- A conclusão deve produzir feedback imediato.
- Sincronizações devem ocorrer sem bloquear a interface.
- Listas extensas devem ser paginadas ou virtualizadas.
- Animações não devem prejudicar dispositivos intermediários.

14.3 Segurança

- Backend permanece como autoridade da progressão.
- Firestore não pode aceitar mutações críticas diretamente do cliente.
- Cada usuário acessa apenas os próprios dados.
- Segredos não podem estar no aplicativo.
- Endpoints críticos devem possuir rate limiting.
- Logs não devem armazenar informações sensíveis.
- Exclusão de conta deve remover ou anonimizar dados conforme a finalidade.

14.4 Privacidade

- Coletar apenas dados necessários.
- Explicar finalidade da coleta.
- Solicitar consentimento quando aplicável.
- Disponibilizar política de privacidade.
- Permitir exclusão e exportação.
- Não utilizar dados pessoais para treinamento de IA sem autorização explícita.

14.5 Acessibilidade

- Suporte a aumento de fonte.
- Contraste adequado.
- Navegação compreensível por leitor de tela.
- Áreas de toque adequadas.
- Feedback não dependente apenas de cor.
- Possibilidade de reduzir animações.
- Textos objetivos.

14.6 Internacionalização

Embora o idioma inicial seja português brasileiro:

- textos não devem ficar hardcoded nos widgets;
 - datas e números devem respeitar locale;
 - estruturas devem permitir novos idiomas;
 - textos de domínio devem utilizar chaves traduzíveis.
-

15. Design do produto

15.1 Direção visual

A identidade deverá transmitir:

- progresso;
- disciplina;
- mistério;
- energia;
- tecnologia;
- evolução.

A interface poderá utilizar:

- fundo escuro;
- painéis de sistema;
- contornos luminosos;
- barras de progresso;
- mapas de Jornada;
- atributos visuais;
- animações curtas;
- tipografia de alto contraste.

Deverá evitar:

- copiar interfaces de animes;
- excesso de neon;
- baixa legibilidade;
- painéis decorativos sem função;
- animações longas em ações frequentes;
- telas carregadas.

15.2 Hierarquia visual

Em cada tela deve existir uma ação principal clara.

Exemplos:

- Base: continuar evolução;
- Missões: executar missão recomendada;
- Jornada: avançar para o próximo marco;
- Ascensão: enfrentar o desafio atual.

15.3 Componentes necessários

O design system deverá incluir:

- botão primário;
- botão secundário;
- botão destrutivo;
- campo de texto;
- seleção de data;
- seleção de recorrência;
- cartão de missão;
- cartão de Jornada;
- cartão de atributo;
- barra de XP;
- barra de vida;
- indicador de Momentum;
- painel de recompensa;
- modal de confirmação;
- estado vazio;
- estado de erro;
- skeleton de carregamento;
- snackbar;
- navegação inferior;
- cabeçalho;
- chips;
- filtros;
- diálogo de retomada.

15.4 Estados obrigatórios

Cada tela deverá possuir definição para:

- carregando;
- vazio;
- conteúdo;
- erro recuperável;
- erro sem conexão;
- sincronização pendente;
- ação concluída;
- permissão negada.

15.5 Movimento

As animações devem:

- confirmar ações;
- representar ganho de progresso;
- durar pouco;
- poder ser reduzidas;
- não impedir navegação;
- não ser obrigatórias em toda conclusão.

15.6 Escrita da interface

A linguagem deve ser:

- direta;
- motivadora;
- não infantilizada;
- não agressiva;
- sem diagnóstico;
- sem culpa;
- sem promessas irreais.

Evitar:

- “Você falhou novamente”.
- “Você perdeu tudo”.
- “Você não tem disciplina”.

Preferir:

- “Seu plano não se ajustou a esta semana”.
- “Vamos reorganizar sua retomada”.
- “Seu progresso permanente foi preservado”.

16. Processo de design

O trabalho de UX/UI deverá produzir:

1. mapa de navegação;
2. fluxos de usuário;
3. wireframes;
4. protótipo navegável;
5. design visual;
6. componentes;

7. estados alternativos;
8. especificações;
9. testes de usabilidade;
10. validação em dispositivo;
11. revisão de acessibilidade;
12. design QA após implementação.

16.1 Fluxos prioritários para prototipação

- onboarding;
- criação de missão;
- conclusão;
- criação de Jornada;
- revisão semanal;
- retomada após ausência;
- desbloqueio de talento;
- Prova de Ascensão;
- assinatura;
- erro de sincronização.

17. Mudanças técnicas

17.1 Domínio proposto

Entidades principais:

```
Usuario
Perfil
Missao
OcorrenciaMissao
RegraRecorrencia
Jornada
Capitulo
Marco
Atributo
ProgressoAtributo
Build
Talentos
TalentosDesbloqueado
Momentum
ChefeSemanal
ContribuicaoChefe
RevisaoSemanal
ProvaAscensao
```

```
Patamar
EventoProgressao
RegistroLegado
Assinatura
```

17.2 Separação de missão e ocorrência

Não tratar uma missão recorrente como um único registro mutável.

```
Missao
- id
- nome
- recorrencia
- atributo
- jornada
- ativa

OcorrenciaMissao
- id
- missaoId
- dataPlanejada
- estado
- dataConclusao
- recompensa
```

17.3 Progressão baseada em eventos

Toda mudança relevante deve gerar um evento canônico.

Exemplos:

```
MissaoCriada
MissaoPlanejada
MissaoConcluida
ConclusaoRevogada
ExperienciaConcedida
AtributoEvoluido
MomentumAtualizado
ChefeDanificado
ChefeDerrotado
JornadaIniciada
CapituloConcluido
ProvaIniciada
```

ProvaConcluida
PatamarConquistado

17.4 Backend autoritativo

O backend deverá ser responsável por:

- validar conclusão;
- calcular XP;
- calcular evolução de atributo;
- atualizar Momentum;
- calcular dano;
- validar talentos;
- validar Provas;
- impedir duplicidade;
- registrar eventos;
- reconciliar estado.

O cliente poderá exibir estado otimista, mas não será autoridade final.

17.5 Offline e sincronização

O cliente deverá possuir:

- cache local;
- fila de comandos pendentes;
- identificador idempotente;
- estado de sincronização;
- política de nova tentativa;
- resolução de conflito;
- reconciliação com estado canônico.

Fluxo:

```
Usuário conclui missão
    ↓
Comando salvo localmente
    ↓
Interface apresenta estado pendente
    ↓
Backend processa comando
    ↓
Resposta canônica recebida
```

↓
Cache reconciliado

17.6 Endpoints sugeridos

Visão geral

```
GET /api/me/overview
GET /api/me/progression
GET /api/me/legacy
```

Missões

```
GET    /api/missions
POST   /api/missions
PUT    /api/missions/{id}
POST   /api/mission-occurrences/{id}/complete
POST   /api/mission-occurrences/{id}/revoke
POST   /api/mission-occurrences/{id}/reschedule
```

Jornadas

```
GET /api/journeys
POST /api/journeys
PUT  /api/journeys/{id}
POST /api/journeys/{id}/pause
POST /api/journeys/{id}/complete
```

Build

```
GET /api/builds
POST /api/me/build
POST /api/me/talents/{id}/unlock
```

Chefe semanal

```
GET /api/weekly-boss
POST /api/weekly-review
```

Ascensão

```
GET /api/ascension-trials
POST /api/ascension-trials/{id}/start
POST /api/ascension-trials/{id}/complete
```

18. Auditoria após a retirada da competição

A competição já foi removida, mas deve existir uma auditoria formal para verificar resíduos.

18.1 Código

Verificar:

- classes sem uso;
- endpoints antigos;
- coleções antigas;
- feature flags;
- serviços de evidência;
- integrações;
- telas inacessíveis;
- modelos;
- DTOs;
- testes;
- configurações;
- permissões.

18.2 Banco e Firestore

Verificar:

- coleções competitivas;
- índices;
- regras de segurança;
- documentos órfãos;
- campos de rank competitivo;
- temporadas;
- evidências;
- quizzes;
- histórico incompatível.

18.3 Documentação

Atualizar:

- README;
- visão;
- roadmap;
- arquitetura;
- backlog;
- diagramas;
- scripts de execução;
- documentação de deploy;
- variáveis de ambiente.

18.4 Infraestrutura

Verificar:

- serviços implantados sem uso;
- Cloud Functions antigas;
- tarefas agendadas;
- secrets;
- permissões;
- custos;
- alertas;
- dashboards.

19. Analytics e métricas

19.1 Eventos de produto

Eventos mínimos:

```
onboarding_iniciado  
onboarding_concluido  
jornada_iniciada  
missao_criada  
missao_concluida  
missao_reagendada  
missao_revogada  
chefe_iniciado  
chefe_derrotado  
revisao_concluida
```

```
retomada_iniciada  
talento_desbloqueado  
prova_iniciada  
prova_concluida  
assinatura_visualizada  
assinatura_iniciada  
assinatura_cancelada  
erro_sincronizacao
```

19.2 Métricas de ativação

- conclusão do onboarding;
- tempo até a primeira missão;
- primeira Jornada iniciada;
- primeira conclusão;
- primeiro retorno.

19.3 Métricas de retenção

- retenção no dia seguinte;
- retenção semanal;
- retenção mensal;
- retorno após ausência;
- missões concluídas por usuário ativo;
- semanas com revisão.

19.4 Métricas do diferencial

- uso de Jornadas;
- interação com build;
- talentos desbloqueados;
- chefes enfrentados;
- Provas iniciadas;
- Provas concluídas;
- acessos ao Legado.

19.5 Métricas de qualidade

- sessões sem falha;
- erros por versão;
- falhas de sincronização;
- tempo de resposta;
- duplicação de recompensa;
- falhas no onboarding;
- falhas de restauração de compra.

20. Estratégia de testes

20.1 Testes de domínio

Cobrir:

- cálculo de XP;
- progressão de nível;
- atributos;
- Momentum;
- dano no chefe;
- recorrência;
- Provas;
- Patamares;
- revogação;
- idempotência.

20.2 Testes de aplicação

Cobrir:

- criação de missão;
- conclusão;
- reagendamento;
- criação de Jornada;
- desbloqueio de talento;
- revisão semanal;
- retomada;
- promoção de Patamar.

20.3 Testes de integração

Cobrir:

- banco;
- Firebase Auth;
- Firestore;
- cache local;
- API;
- fila offline;
- reconciliação;
- assinatura.

20.4 Testes de interface

Cobrir:

- onboarding;
- navegação;
- formulários;
- estados vazios;
- estados de erro;
- feedback de conclusão;
- acessibilidade;
- diferentes tamanhos de tela.

20.5 Testes de contrato

Garantir compatibilidade entre:

- Flutter;
- API Java;
- serialização;
- enums;
- códigos de erro;
- versionamento.

20.6 Testes manuais em dispositivo

Cenários mínimos:

- rede estável;
 - rede lenta;
 - modo avião;
 - encerramento abrupto;
 - atualização de versão;
 - troca de conta;
 - reinstalação;
 - sessão expirada;
 - compra restaurada;
 - notificações negadas.
-

21. Épicos do projeto

Épico 1 — Alinhamento de produto

Objetivo

Estabelecer uma única visão para toda a equipe.

Entregas

- visão atualizada;
- glossário;
- escopo;
- requisitos;
- arquitetura da informação;
- mapa de dependências;
- critérios de sucesso;
- ADR da nova direção.

Critério de conclusão

Não devem existir documentos ativos que apresentem competição como parte do produto.

Épico 2 — Auditoria técnica

Objetivo

Garantir que a retirada da competição não deixou resíduos ou inconsistências.

Entregas

- inventário do código;
- inventário de infraestrutura;
- inventário de dados;
- lista de remoções;
- migrações;
- atualização de testes;
- atualização de documentação.

Critério de conclusão

Nenhum fluxo de produção deve depender de estruturas competitivas.

Épico 3 — Design system e navegação

Objetivo

Criar a base visual e estrutural da nova experiência.

Entregas

- navegação de quatro áreas;
- tokens visuais;
- componentes;
- protótipos;
- estados;
- acessibilidade;
- diretrizes de animação.

Critério de conclusão

Todos os fluxos P0 devem estar representados em protótipo navegável.

Épico 4 — Onboarding e ativação

Objetivo

Fazer o usuário experimentar valor rapidamente.

Entregas

- onboarding curto;
- primeira Jornada;
- primeiras missões;
- persistência;
- retomada;
- feedback inicial.

Critério de conclusão

O usuário consegue chegar ao primeiro ganho de XP sem assinatura e sem questionário extenso.

Épico 5 — Missões e recorrência

Objetivo

Criar uma base confiável para o uso diário.

Entregas

- CRUD;
- recorrência;
- ocorrências;
- lembretes;
- reagendamento;
- pausa;
- histórico;
- conclusão offline.

Critério de conclusão

Missões recorrentes funcionam sem duplicar, perder ou corromper ocorrências.

Épico 6 — Progressão pessoal

Objetivo

Tornar cada ação significativa.

Entregas

- XP;
- níveis;
- atributos;
- eventos;
- feedback;
- próximo desbloqueio;
- proteções contra exploração.

Critério de conclusão

Toda recompensa pode ser explicada e rastreada até uma ação válida.

Épico 7 — Build e talentos

Objetivo

Permitir que o usuário tome decisões sobre o personagem.

Entregas

- três builds;
- árvores iniciais;

- pontos;
- efeitos;
- desbloqueios.

Critério de conclusão

Usuários com histórico semelhante podem possuir builds diferentes.

Épico 8 — Jornadas

Objetivo

Conectar tarefas a objetivos reais.

Entregas

- Jornada;
- capítulos;
- marcos;
- modelos;
- progresso;
- conclusão;
- histórico.

Critério de conclusão

O usuário consegue identificar qual objetivo está sendo beneficiado por cada missão.

Épico 9 — Chefe e revisão semanal

Objetivo

Criar tensão e fechamento semanal.

Entregas

- chefe;
- vida;
- dano;
- recompensas;
- derrota;
- revisão;
- replanejamento.

Critério de conclusão

Uma semana produz início, progresso, resultado e ajuste.

Épico 10 — Recuperação

Objetivo

Evitar abandono após falhas.

Entregas

- detecção de ausência;
- retomada leve;
- reorganização;
- limpeza de pendências;
- ajuste de Momentum.

Critério de conclusão

O usuário consegue retornar sem lidar manualmente com uma lista extensa de atrasos.

Épico 11 — Ascensão e Legado

Objetivo

Criar progressão permanente de médio prazo.

Entregas

- Provas;
- Patamares;
- Legado;
- títulos;
- histórico.

Critério de conclusão

O usuário possui conquistas permanentes que representam sua trajetória.

Épico 12 — Monetização

Objetivo

Criar receita sem quebrar confiança.

Entregas

- plano gratuito;
- plano Premium;
- entitlement;
- tela de assinatura;
- restauração;
- cancelamento;
- analytics.

Critério de conclusão

O usuário entende o valor pago antes de iniciar a compra.

Épico 13 — Produção

Objetivo

Preparar uma versão estável para usuários reais.

Entregas

- observabilidade;
- Crashlytics;
- Analytics;
- suporte;
- privacidade;
- exclusão;
- staging;
- produção;
- testes;
- lojas;
- beta.

Critério de conclusão

Todos os requisitos P0 funcionam em dispositivo real e possuem monitoramento.

22. Sequenciamento recomendado

Fase 1 — Fundação

- alinhamento;
- auditoria;
- arquitetura da informação;
- design system;
- modelo de dados;
- estratégia de sincronização.

Fase 2 — Loop diário

- onboarding;
- missões;
- recorrência;
- conclusão;
- offline;
- histórico;
- notificações.

Fase 3 — Progressão

- XP;
- nível;
- atributos;
- Momentum;
- build;
- talentos.

Fase 4 — Objetivos

- Jornadas;
- capítulos;
- marcos;
- modelos;
- progresso.

Fase 5 — Desafios

- chefe semanal;
- revisão;
- recuperação;
- Provas;
- Patamares;
- Legado.

Fase 6 — Produção

- monetização;
- suporte;
- privacidade;
- observabilidade;
- beta;
- correções;
- publicação.

23. Dependências principais

Entrega	Dependência
Progressão	Eventos de conclusão
Chefe semanal	Missões e progressão
Jornada	Missões vinculáveis
Build	Progressão e atributos
Provas	Progressão, Jornada e chefe
Patamares	Provas
Legado	Eventos canônicos
Monetização	Entitlements
Analytics	Taxonomia de eventos
Offline	Modelo de comandos idempotentes

24. Organização da equipe

Produto

Responsável por:

- visão;
- requisitos;
- prioridades;
- métricas;
- validação de hipóteses;
- aceite funcional.

UX/UI

Responsável por:

- pesquisa;
- fluxos;
- protótipos;
- design system;
- acessibilidade;
- design QA.

Flutter

Responsável por:

- interfaces;
- estado;
- cache;
- navegação;
- offline;
- sincronização;
- notificações;
- integração com lojas.

Backend

Responsável por:

- domínio;
- API;
- progressão;
- idempotência;
- segurança;
- banco;
- observabilidade.

Qualidade

Responsável por:

- estratégia de testes;
- casos;
- regressão;
- dispositivos;
- critérios de saída;
- automação.

Infraestrutura

Responsável por:

- ambientes;
 - CI/CD;
 - segredos;
 - monitoramento;
 - custos;
 - deploy;
 - recuperação.
-

25. Definition of Ready

Uma história estará pronta para desenvolvimento quando possuir:

- objetivo;
 - descrição;
 - prioridade;
 - fluxo;
 - critérios de aceite;
 - estados de erro;
 - design;
 - regras de negócio;
 - dependências;
 - eventos de analytics;
 - impacto offline;
 - impacto de segurança;
 - plano de teste.
-

26. Definition of Done

Uma história será considerada concluída quando:

- código estiver revisado;
- testes automatizados passarem;
- critérios de aceite estiverem validados;
- estados de erro estiverem implementados;
- analytics estiver instrumentado;
- documentação estiver atualizada;
- acessibilidade estiver revisada;
- design QA estiver aprovado;

- não houver regressão conhecida P0;
 - funcionalidade estiver validada em dispositivo.
-

27. Backlog inicial recomendado

Prioridade 1 — Baseline

1. Atualizar visão e escopo.
2. Criar glossário.
3. Auditar resíduos competitivos.
4. Definir entidades.
5. Definir eventos.
6. Definir navegação.
7. Criar design system inicial.
8. Definir estratégia offline.

Prioridade 2 — Ativação

1. Reformular onboarding.
2. Persistir progresso por etapa.
3. Criar primeira Jornada.
4. Criar primeiras missões.
5. Permitir edição antes da ativação.
6. Implementar primeira conclusão.
7. Exibir feedback de XP.

Prioridade 3 — Loop diário

1. Separar missão e ocorrência.
2. Implementar recorrência.
3. Implementar reagendamento.
4. Implementar pausa.
5. Implementar histórico.
6. Implementar conclusão offline.
7. Implementar reconciliação.

Prioridade 4 — Progressão

1. Consolidar XP.
2. Consolidar atributos.
3. Implementar Momentum.
4. Implementar builds.
5. Implementar talentos.
6. Implementar próximo desbloqueio.

Prioridade 5 — Diferenciais

1. Jornadas completas.
2. Chefe semanal.
3. Revisão.
4. Retomada.
5. Provas.
6. Patamares.
7. Legado.

Prioridade 6 — Produção

1. Assinatura.
2. Suporte.
3. Exclusão.
4. Analytics.
5. Crash reporting.
6. Observabilidade.
7. Beta.
8. Publicação.

28. Riscos e mitigação

Risco	Mitigação
Produto tornar-se complexo	Exposição progressiva de funcionalidades
RPG tornar-se decorativo	Exigir consequência para cada sistema
Usuário explorar recompensas	Regras autoritativas e limites
Onboarding longo	Coleta mínima e persistência
Dependência de IA	Regras determinísticas
Interface parecer cópia	Identidade visual original
Falhas de sincronização	Outbox, idempotência e reconciliação
Escopo impedir lançamento	Separação entre P0, P1 e P2
Paywall gerar rejeição	Plano gratuito útil e transparência
Ausência provocar abandono	Fluxo de recuperação
Código competitivo residual	Auditoria formal
Animações prejudicarem uso	Duração curta e redução opcional

29. Critérios para a primeira versão de produção

O Ascend estará apto para produção quando um usuário puder:

1. criar uma conta;
2. concluir o onboarding;
3. iniciar uma Jornada;
4. criar e editar missões;
5. utilizar recorrências;
6. concluir atividades offline;
7. sincronizar posteriormente;
8. ganhar XP;
9. desenvolver atributos;
10. selecionar uma build;
11. desbloquear talentos;
12. acompanhar Momentum;
13. enfrentar um chefe semanal;
14. revisar a semana;
15. retornar após uma ausência;
16. concluir uma Prova;
17. avançar de Patamar;
18. consultar seu Legado;
19. utilizar continuamente o plano gratuito;
20. excluir sua conta.

Todos esses fluxos deverão funcionar sem:

- competição;
- ranking;
- evidência obrigatória;
- câmera;
- Health Connect;
- IA obrigatória;
- pagamento para iniciar;
- solicitação de avaliação precoce.

30. Resultado esperado

Ao final da renovação, o Ascend não será apresentado como um aplicativo de tarefas com elementos de RPG.

Ele será um sistema de evolução pessoal composto por:

- ações diárias simples;

- objetivos estruturados;
- progressão compreensível;
- escolhas de personagem;
- desafios pessoais;
- recuperação;
- trajetória permanente.

A principal promessa será:

No Ascend, cada ação real deixa uma marca visível na evolução do seu personagem e na história da sua Jornada.